

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

Ankith U Davey 24011101004

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness

- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

Logic Gate Datasets (AND, OR, NOT)

In addition to the banknote dataset, the same perceptron implementation was tested on the OR, AND, and NOT logic gates.

OR Gate

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

AND Gate

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

NOT Gate

x_1	y
0	1
1	0

5. Source Code

The complete source code is available in the accompanying GitHub repository: <https://github.com/Ankith-Davey/Deep-Learning-Lab-Ankith-24011101004/tree/main/Lab%201>

6. Experimental Results

The perceptron was trained for 50 epochs at a learning rate of 0.01. The training error decreased sharply during the first 10 epochs, then it plateaued into a noisy oscillation between 8 and 19 misclassified samples and then ending at 13 misclassified samples at epoch 50. The the model did not fully converge to zero error, which indicates that the two classes are not perfectly linearly separable in this 4-feature space and at this train/test split. On the test set, the SLP achieved 97.82% accuracy.

The same perceptron was also evaluated on the OR, AND, and NOT logic gates, each of which converged within a few weight updates, as detailed in the Conclusion.

7. Generated Plots with Interpretation

Feature Histograms

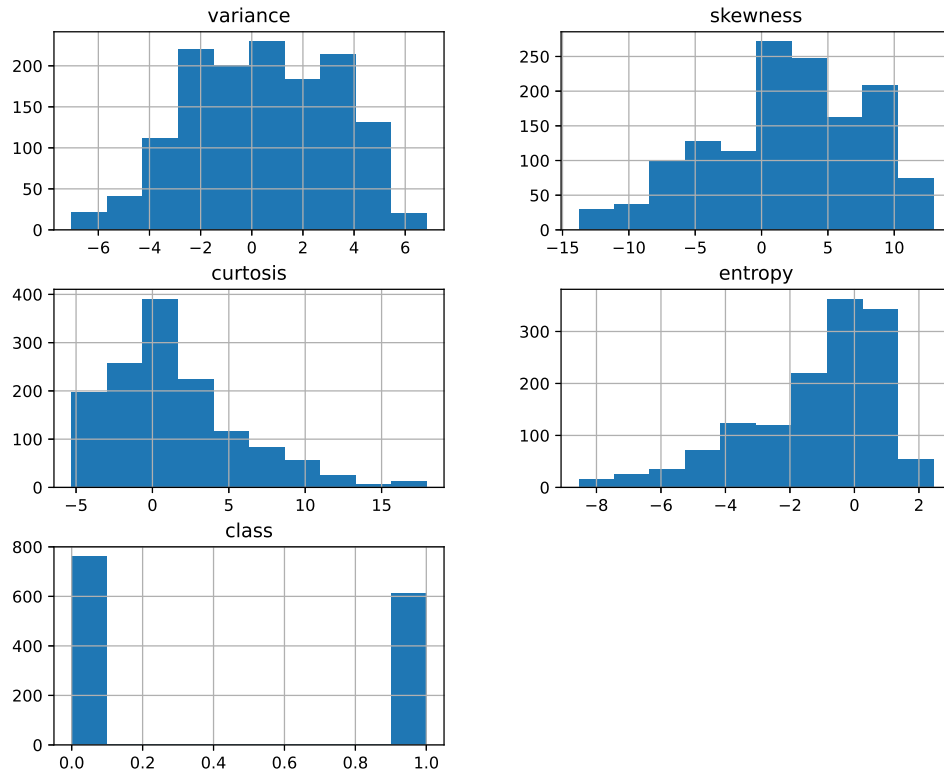


Figure 1: Feature Histograms: distribution of Variance, Skewness, Kurtosis, and Entropy, and the class balance, across all 1372 banknote samples.

The variance shows a symmetric distribution, while Skewness is bimodal with two distinct peaks but mostly towards the right side. Kurtosis gives a right-skewed graph with a long tail towards higher values, and Entropy is left-skewed with majorly clustering near its upper range with a tail towards the negative values.

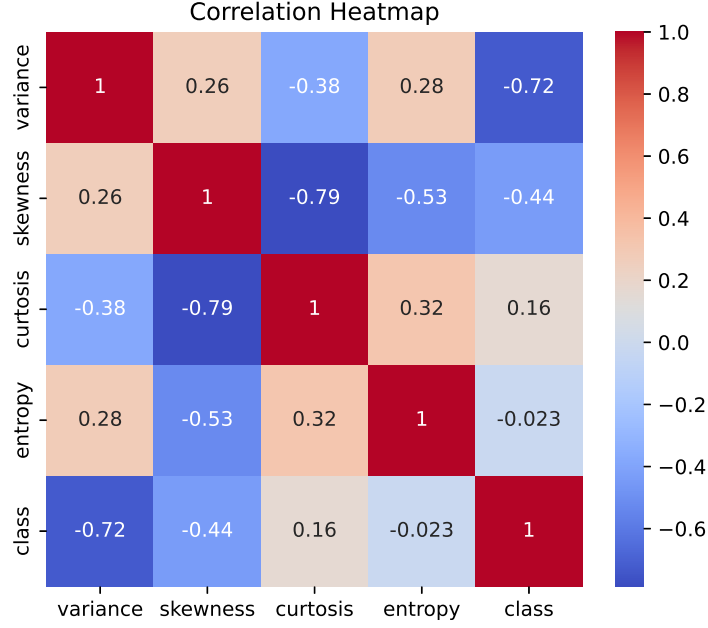


Figure 2: Correlation Heatmap showing pairwise Pearson correlation coefficients between the four input features and the class label.

Skewness and kurtosis shows a strong correlation between each other (-0.79), which suggests that there is some redundancy between them. Variance has the strongest relationship with the class label with a value of -0.72 which makes it the most informative/important feature for classification. Meanwhile Entropy is the least correlated to the class label (-0.02).

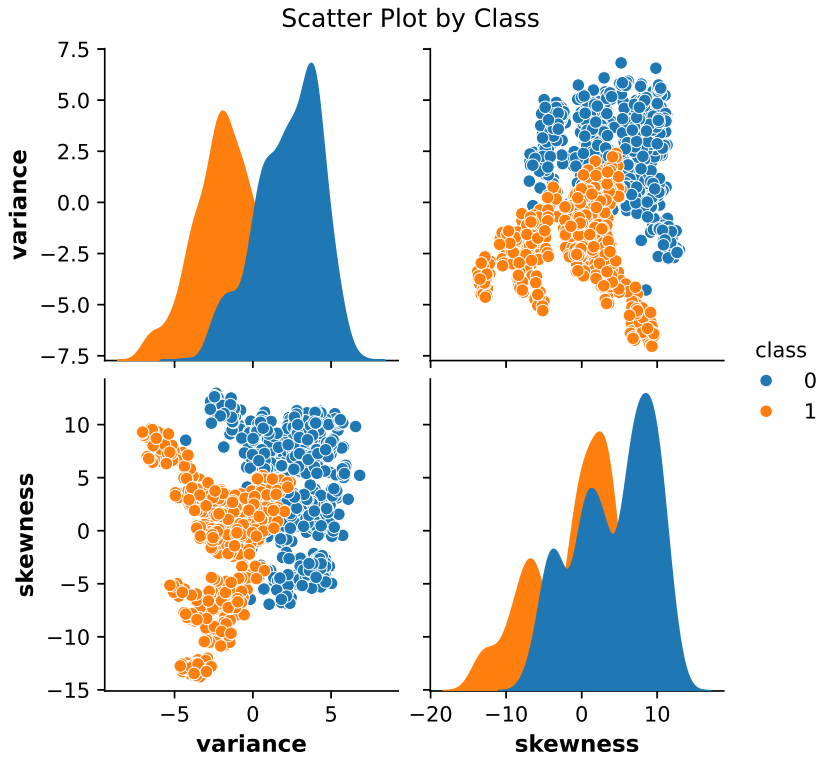


Figure 3: Scatter Plot of Variance vs. Skewness, colored by class, used to visually assess linear separability of the two classes.

This plot of Variance against Skewness by class shows mainly two distinguishable clusters,

but with a small region of overlap between them. The overlap indicates that the two classes are not perfectly linearly separable using just variance and skewness.

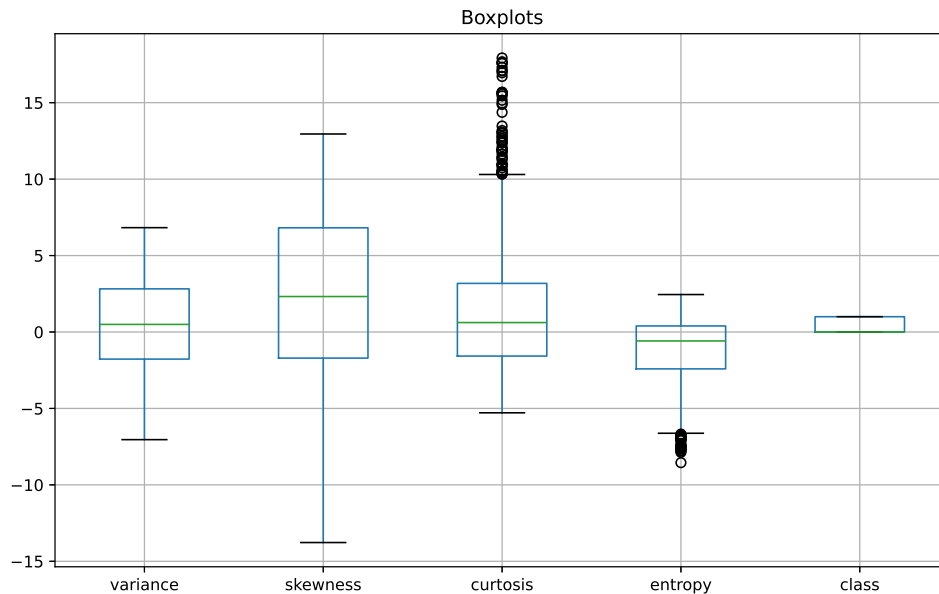


Figure 4: Boxplots of all four features, used to compare feature spread and identify outliers.

In the boxplot the kurtosis feature shows the most outliers, with several points extending beyond its upper whisker. Variance and skewness show clear distributions with 0 visible outliers. But Entropy has a smaller cluster of outliers comparatively.

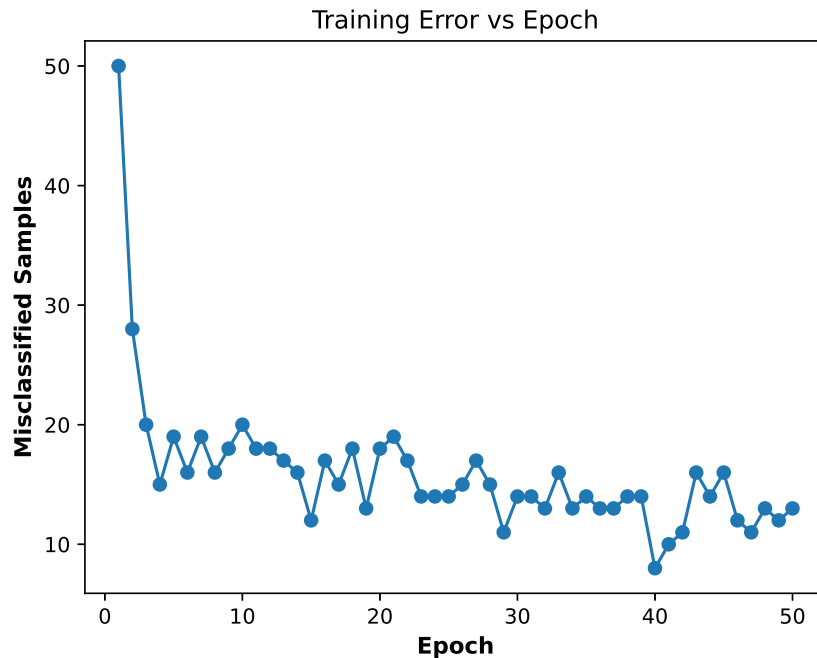


Figure 5: Number of misclassified training samples at the end of each epoch, showing the perceptron's convergence behaviour over 50 epochs.

The graph displays how the number of misclassified samples reduced as the number of epochs increased. But it also says that the error did not become zero even after the 50th epoch which implies that the model is still learning.

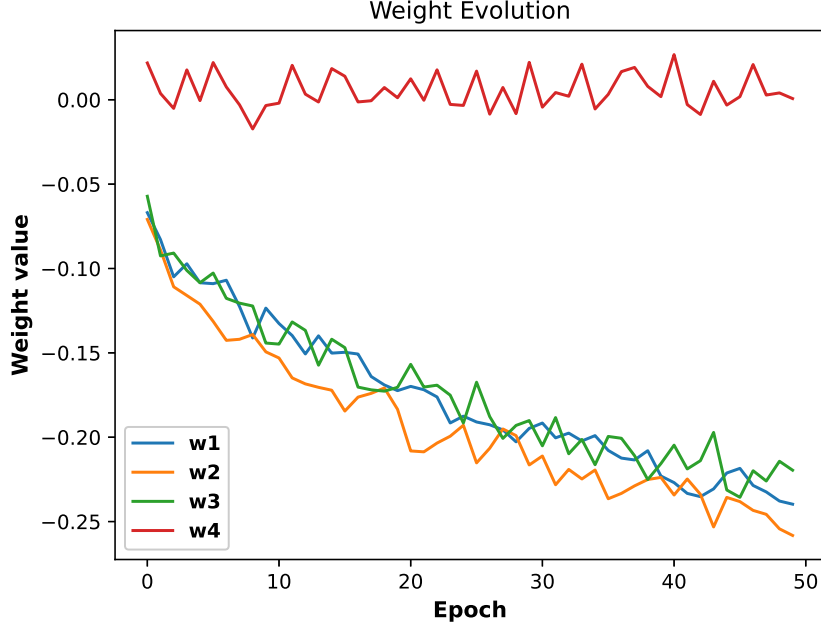


Figure 6: Evolution of all four model weights across training epochs.

It represents how the 4 weight values are changing with respect to the number of epochs in case of a single layer perceptron. For the given dataset, the first 3 weights negatively decrease which indicates there is a small delta change for these weights but the interesting thing is that the entropy's (4th feature) weight does not change much even with increase in epochs and is almost near zero and this is further supported by the correlation heat map as entropy feature is least related to class label with its value being almost equal to 0.

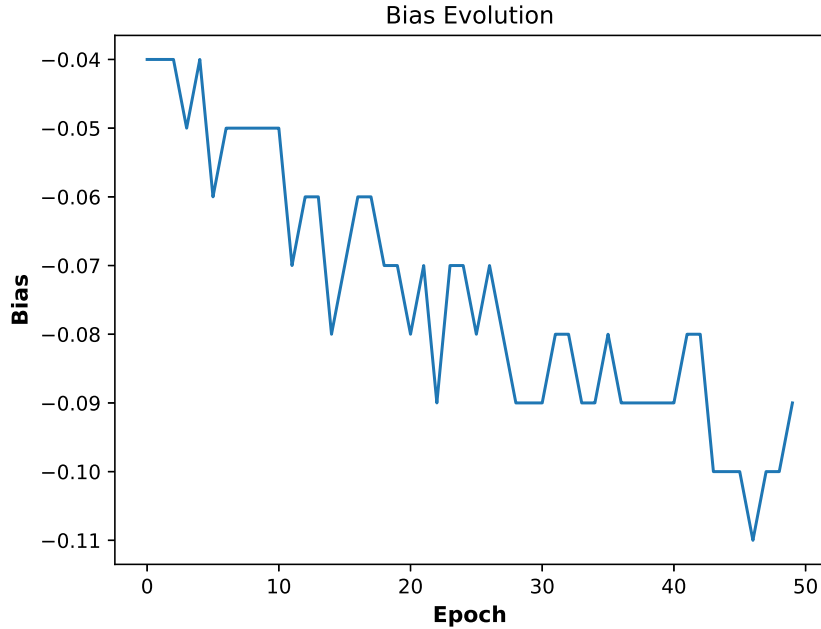


Figure 7: Evolution of the model's bias term across training epochs.

The bias term moved from 0 to -0.09 after 50 epochs, which implies that the model's learning did not complete with the non-zero training error observed throughout the training.

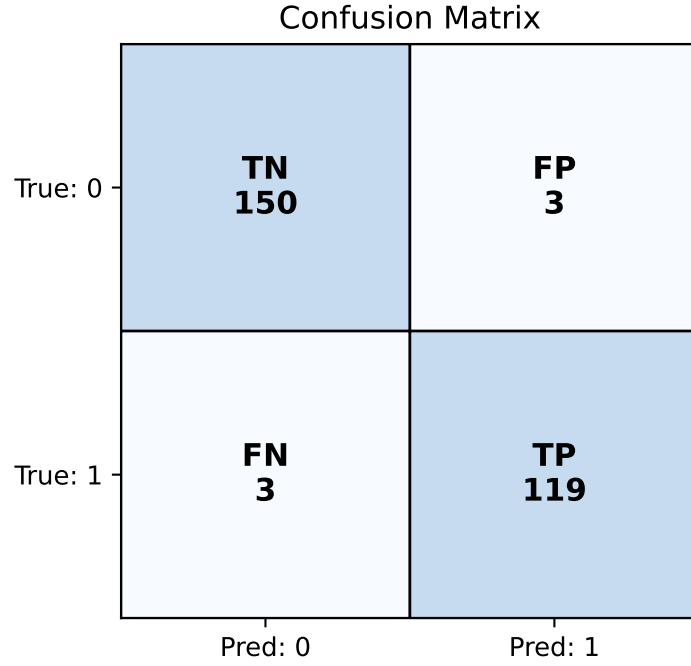


Figure 8: Confusion matrix on the held-out test set (N=275), showing true/false positives and negatives.

Of 275 test samples, 269 were classified correctly. The SLP gave only 3 false positives and 3 false negatives.

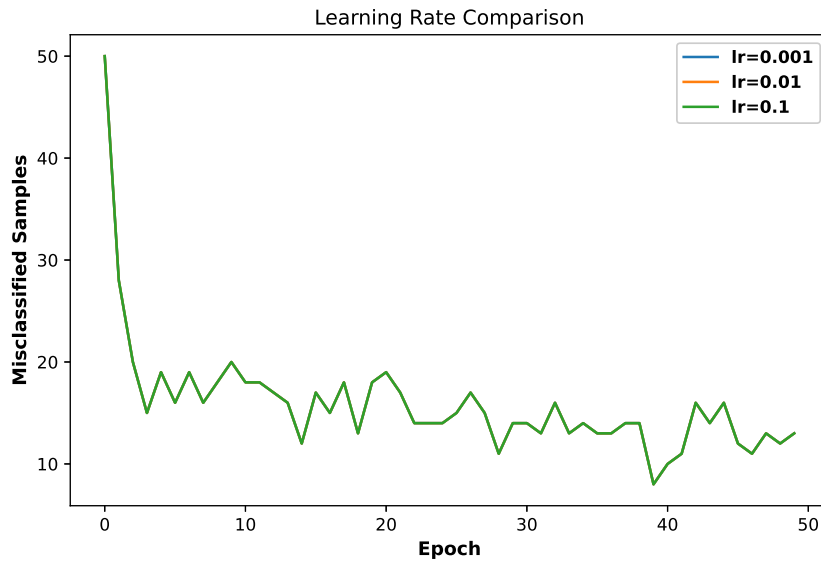


Figure 9: Misclassified samples per epoch for learning rates 0.001, 0.01, and 0.1, compared on the same axes.

All the three learning rates appear to converge into a similar curve which suggests that the model's performance ceiling is driven by class overlapping in the data rather than the learning rate values.

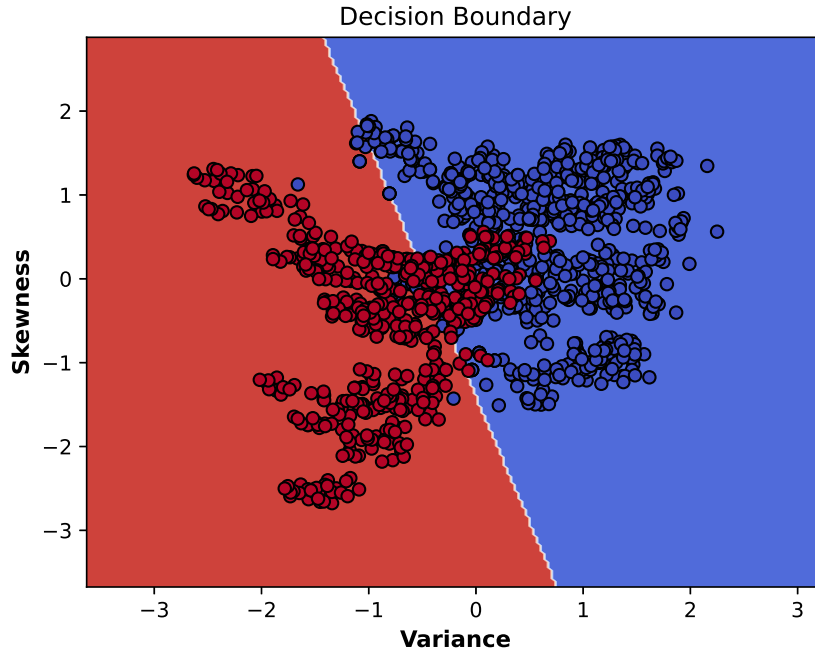


Figure 10: Decision boundary of a 2-feature perceptron trained on Variance and Skewness, with the true data points overlaid.

The decision boundary plotted in 2D using variance and skewness displays that the bank notes are mostly classified correctly except for a few misclassified ones which are due to representation using only 2 features rather than 4 features.

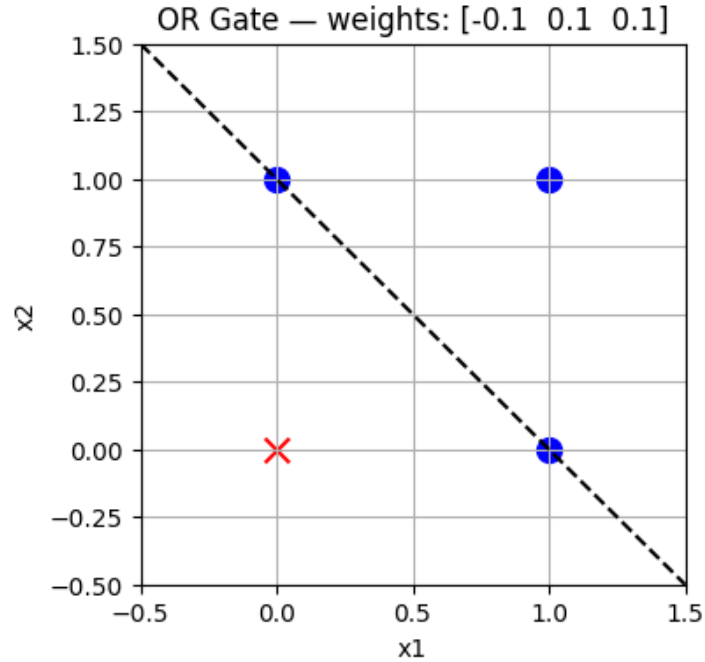


Figure 11: Final decision boundary of the perceptron trained on the OR gate.

The OR gate converged at the 3rd epoch with final weight values being $w_1 = 0.1$, $w_2 = 0.1$, and bias equal to -0.1 .

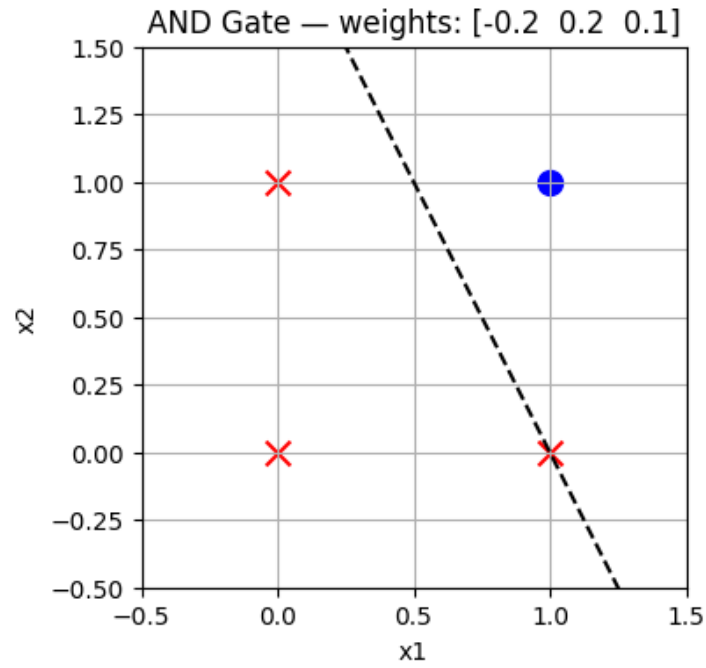


Figure 12: Final decision boundary of the perceptron trained on the AND gate.

The AND gate converged at the 3rd epoch with final weight values being $w_1 = 0.2$, $w_2 = 0.1$, and bias equal to -0.2 .

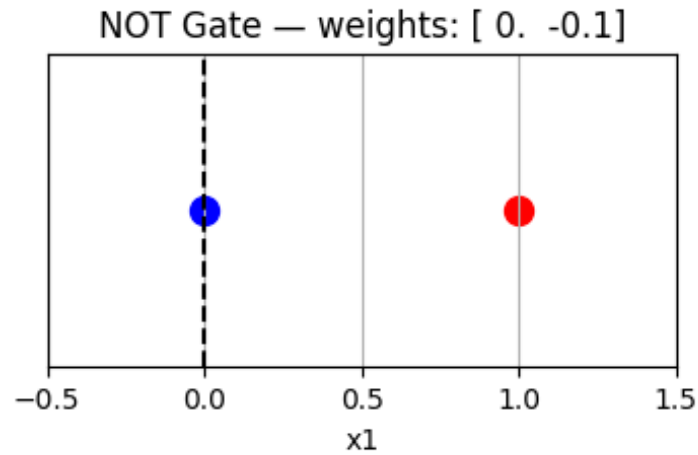


Figure 13: Final decision boundary of the perceptron trained on the NOT gate.

The NOT gate converged at the 2nd epoch with final weight value being $w_1 = -0.1$, and bias equal to 0.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275 (80/20, stratified)
Learning Rate	0.01
Epochs	50
Final Weights	[-0.240, -0.258, -0.220, 0.001]
Final Bias	-0.090
Accuracy	0.9782
Precision	0.9754
Recall	0.9754
F1-score	0.9754

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	50	-0.067	-0.071	-0.040
10	20	-0.123	-0.149	-0.050
20	18	-0.172	-0.183	-0.070
30	14	-0.195	-0.216	-0.090
40	8	-0.223	-0.224	-0.090
50	13	-0.240	-0.258	-0.090

Confusion Matrix (Test Set, N=275)

		Predicted	
		0 (Authentic)	1 (Forged)
Actual	0	150	3
	1	3	119

Logic Gate Convergence Summary

Gate	Updates to Converge	Final w_1	Final w_2	Final Bias
OR	5	0.1	0.1	-0.1
AND	8	0.2	0.1	-0.2
NOT	2	-0.1	—	0.0

9. Additional Tasks

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

The Step function gives only binary outputs and 0 gradient, which makes it non-differentiable at the decision boundary and thus prevents the model from learning and updating weights during backpropagation. But on the other hand, the Sigmoid function's output values range between 0 and 1 which also implies that it is differentiable thus enabling gradient descent to update and modify weights according to the error. This is the reason why modern deep

learning networks use Sigmoid, ReLU, or other similar differentiable functions rather than the Step function.

2. Compare the implementation with Scikit-learn's Perceptron.

The SLP's implementation from scratch achieved 97.82% accuracy on the test set when compared to scikit-learn's Perceptron which achieved an accuracy of 98.18% under identical train/test splits and learning rate. The difference is due to the differences in the internal convergence criteria and other default regularizations which is handled in scikit-learn's implementation. Both the manual SLP implementation and the scikit-learn's implementation follow the same mathematical formulation.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

All the three learning rates appear to converge into a similar curve, which suggests that the model's performance is driven by the class overlap in the data rather than the specific learning rate value being chosen.

4. Explain why a Single Layer Perceptron cannot solve the XOR problem.

XOR is not linearly separable which means that no single straight line (or hyperplane) can act as a decision boundary to separate the 4 points. Since a single-layer perceptron can only learn a single linear decision boundary, it cannot represent the XOR function irrespective of the weights chosen. The XOR problem can be solved by a Multilayer Perceptron which is capable of solving non-linear decision boundary problems.

5. Study the effect of feature normalization on model convergence.

Feature normalization methods like standardizing features to zero mean and unit variance ensures that no single feature dominates the weighted sum due to difference in scales. It also helps to speed up the convergence of the perceptron learning rule. Without this normalization, features like Skewness in this dataset would disproportionately influence the weighted sum simply because it has a large range of values compared to smaller-range features like Entropy.

10. Discussion

The Single Layer Perceptron performed strongly overall, it correctly classified 269 out of 275 test samples, with precision and recall both equal to 0.9754 which means that the number of true classifications were far more than the false ones. But even after 50 epochs, the training never fully converged and this can be supported by the training error vs epoch graph wherein the training error dropped steadily but never reached zero, and the weight and bias plots also show the same pattern. If we trace this back to the data it explains why the scatter plot shows a small region wherein variance and skewness values overlap between the 2 classes and the 2D decision boundary confirms this using those two features alone (variance and skewness), though the actual 4-feature model's own training error is a more direct evidence that the model doesn't perfectly converge. Regarding the features, it stayed consistent across unrelated plots, that is, Variance was the most useful feature by its correlation with the class label, and correspondingly it was one of the most active weights during training, while Entropy barely correlated with the label and its weight barely changed. So, while a few samples were misclassified, the model's overall performance shows a single-layer perceptron is a strong fit for the given dataset.

The banknote SLP had never fully converged because of feature overlap but on the other hand the AND, OR and NOT gates converged in a few updates because the dataset for the logic gates is perfectly linearly separable. The NOT gate has only a single input, so the "boundary" collapses to a threshold point rather than a line/hyperplane.

11. Conclusion

This experiment involved implementing a Single Layer Perceptron from scratch to classify banknotes as forged or real, which showed that the SLP performed well, reaching 97.82% accuracy. But the training didn't fully converge, since variance and skewness overlapped slightly between the two classes which meant that no single straight-line boundary separates them with zero error, but this affected only a small fraction of samples and didn't impact much on the overall performance. This non-converging behavior also implies the perceptron's limitation on data that isn't perfectly linearly separable, which is the same limitation due to which deep learning models use differentiable activations and multiple hidden layers on non-linearly separable data, though for this dataset, which is almost linearly separable, the SLP proves to be a good model.

In addition to the banknote dataset, the same perceptron implementation was also tested on the OR, AND, and NOT logic gates. The OR gate converged in 5 weight updates, but AND gate required 8 updates and the NOT gate converged in only 2 updates. Unlike the banknote dataset, the three logic gates reached 0 training error within a few updates, this is because the datasets are perfectly linearly separable. However, the XOR gate cannot be linearly separated using a single-layer perceptron, and hence a multi-layer perceptron (MLP) is required to separate the datapoints.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.